

Реферат

Обзор методов защиты файлов в операционных системах

Лащиков Алексей Антонович

Содержание

1	Введение	2
2	Теоретические основы защиты файлов	2
2.1	Базовые свойства безопасности	2
2.2	Модели управления доступом	4
3	Контроль доступа к файлам	5
3.1	Традиционные права доступа и ACL	5
3.2	Обязательные ограничения и песочницы	7
3.3	Принятие решения о доступе	7
4	Шифрование как метод защиты файлов	8
4.1	Назначение шифрования	8
4.2	Полное и файловое шифрование	8
4.3	Аппаратная поддержка ключей	10
5	Контроль целостности и аутентичности	11
5.1	Проверка целостности данных	11
5.2	Подписи и доверие к исполняемым файлам	11
6	Аудит и журналирование	11
7	Безопасное удаление и остаточные данные	12
8	Сравнение подходов в различных операционных системах	12
9	Проблемы и ограничения существующих методов	13
10	Заключение	14
	Список литературы	14

1. Введение

Защита файлов в операционных системах является одной из базовых задач информационной безопасности. Именно файл выступает основной единицей хранения пользовательских документов, системных настроек, журналов, программ и служебных данных. Нарушение конфиденциальности, целостности или доступности файлов может привести к утечке информации, искажению данных, нарушению работы программ и компрометации всей системы.

Современные операционные системы не ограничиваются одним средством защиты. На практике используется многоуровневый подход: сначала система определяет пользователя и его полномочия, затем проверяет правила доступа к объекту, при необходимости применяет шифрование, средства контроля целостности, журналирование и механизмы безопасного удаления данных ([National Institute of Standards and Technology 2025a](#), [2025b](#); [Microsoft 2025d](#); [Chandramouli et al. 2025](#)). Общая логика такого подхода показана на [рис. 1](#).

Цель данного реферата — рассмотреть основные методы защиты файлов в операционных системах, показать их назначение, достоинства и ограничения, а также сопоставить подходы, применяемые в разных семействах ОС.

2. Теоретические основы защиты файлов

2.1. Базовые свойства безопасности

В контексте файловой защиты обычно рассматриваются три классических свойства информации: конфиденциальность, целостность и доступность. Конфиденциальность означает, что данные могут читать только уполномоченные субъекты. Целостность предполагает невозможность незаметного изменения файла или, по крайней мере, возможность обнаружить такое изменение. Доступность означает, что легальный пользователь или сервис может получить доступ к данным тогда, когда это требуется. В нормативной и эксплуатационной документации по защищённым ОС подчёркивается, что политика безопасности должна учитывать не только права пользователя, но и состояние системы, правила разграничения доступа, контроль целостности и регистрацию событий безопасности ([Astra Linux Wiki 2021](#); [Astra Linux 2026c](#)).

Файловая защита опирается на понятия субъекта и объекта безопасности. Субъектом считается активная сущность, которая инициирует действие, например пользователь или процесс. Объектом выступает то, к чему производится обращение: файл, каталог, устройство или иной ресурс. В UNIX-подобных системах субъектами доступа являются пользователи и

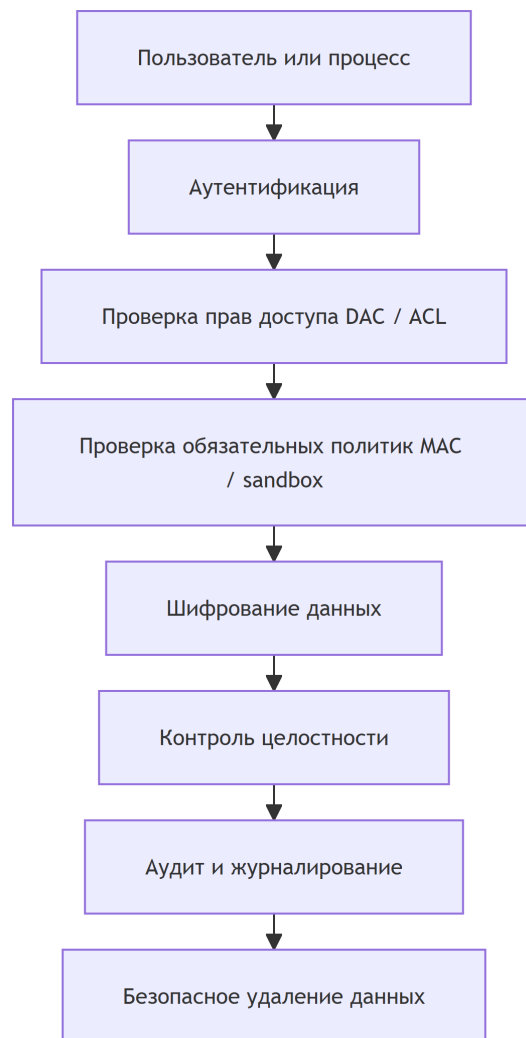


Рисунок 1: Основные уровни защиты файлов в операционной системе

процессы, а объектами — файлы, каталоги и другие сущности файловой системы; именно на этом строятся как дискреционные, так и мандатные механизмы защиты ([Astra Linux 2026a, 2026b](#)).

2.2. Модели управления доступом

В основе большинства средств защиты файлов лежат формальные модели управления доступом. Наиболее известными являются дискреционная модель DAC, мандатная модель MAC, ролевая модель RBAC и атрибутная модель ABAC.

Дискреционное управление доступом, или DAC, предполагает, что субъект, уже имеющий определённые права на объект, в рамках правил системы может передавать их другим субъектам или изменять параметры доступа ([National Institute of Standards and Technology 2025a](#)). Такая модель исторически лежит в основе файловых прав UNIX и NTFS ACL в Windows. Её достоинство заключается в простоте и гибкости, однако она сильно зависит от корректности действий владельца объекта.

Мандатное управление доступом, или MAC, строится на централизованной политике, которая применяется ко всем субъектам и объектам единообразно. Решение о доступе определяется не желанием владельца файла, а набором меток и правил безопасности ([National Institute of Standards and Technology 2025b](#)). Именно этот подход используется в системах SELinux, AppArmor, Mandatory Integrity Control и ряде песочниц.

RBAC связывает права не напрямую с пользователями, а с ролями. Пользователь получает полномочия в зависимости от роли, которую выполняет в системе: например, администратор, оператор, аудитор или обычный сотрудник ([National Institute of Standards and Technology 2025c](#)). Такая модель особенно удобна в корпоративных инфраструктурах.

ABAC делает решение о доступе ещё более гибким. Оно вычисляется с учётом атрибутов субъекта, объекта, операции и окружения: времени, места, типа устройства или состояния сеанса ([Hu et al. 2019](#)). Хотя в классических настольных ОС ABAC применяется ограниченно, его идеи активно используются в корпоративных системах управления доступом и облачных средах.

На практике операционные системы редко используют только одну модель. Обычно базовые файловые права строятся на DAC, а поверх них добавляются обязательные ограничения MAC, изоляция процессов и правила журналирования. Поэтому защита файла почти всегда является результатом совместной работы нескольких механизмов.

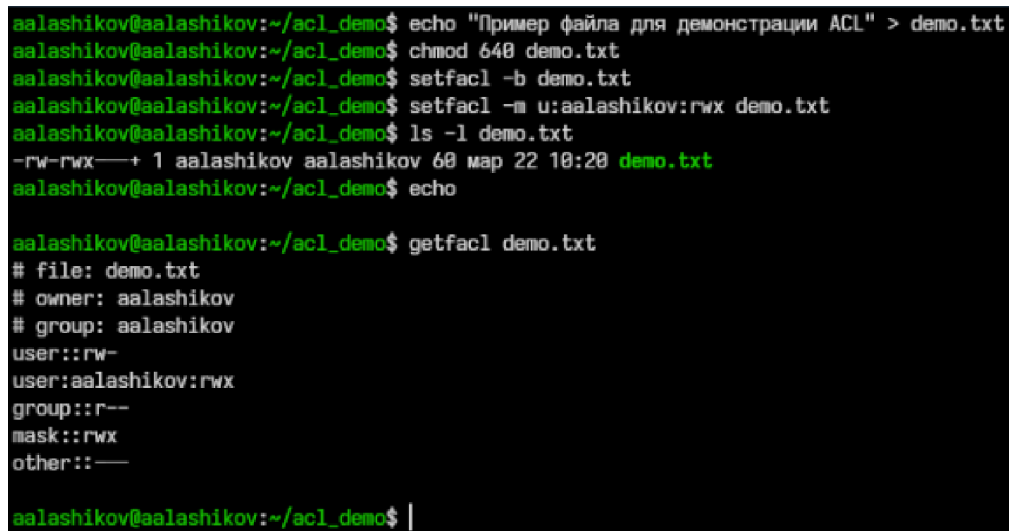
3. Контроль доступа к файлам

3.1. Традиционные права доступа и ACL

Самым известным методом защиты файлов являются права доступа. В UNIX-подобных ОС для каждого файла и каталога задаются права чтения, записи и выполнения для владельца, группы владельца и остальных пользователей ([Astra Linux 2026a](#)). Такая модель проста и удобна, но её возможностей недостаточно, когда требуется задавать разные комбинации разрешений для большого числа пользователей и групп. Поэтому современные файловые системы дополняют базовые права списками контроля доступа ACL ([Astra Linux Wiki 2026](#); [Базальт СПО 2025](#)).

POSIX ACL расширяют традиционную модель и позволяют назначать права конкретным пользователям и группам намного гибче, чем обычная схема owner/group/other ([Astra Linux Wiki 2026](#); [Базальт СПО 2025](#)). При этом важным элементом является маска ACL, ограничивающая эффективные права для именованных записей. Неправильная настройка маски нередко приводит к ситуациям, когда формально право выдано, но фактически не действует ([Базальт СПО 2024, 2025](#)).

На [рис. 2](#) показан пример просмотра стандартных прав доступа и расширенных ACL для файла в Linux.



```
aalashikov@aalashikov:~/acl_demo$ echo "Пример файла для демонстрации ACL" > demo.txt
aalashikov@aalashikov:~/acl_demo$ chmod 640 demo.txt
aalashikov@aalashikov:~/acl_demo$ setfacl -b demo.txt
aalashikov@aalashikov:~/acl_demo$ setfacl -m u:aalashikov:rwx demo.txt
aalashikov@aalashikov:~/acl_demo$ ls -l demo.txt
-rw-rwx--+ 1 aalashikov aalashikov 60 map 22 10:20 demo.txt
aalashikov@aalashikov:~/acl_demo$ echo

aalashikov@aalashikov:~/acl_demo$ getfacl demo.txt
# file: demo.txt
# owner: aalashikov
# group: aalashikov
user::rw-
user:aalashikov:rwx
group::r--
mask::rwx
other::---
```

Рисунок 2: Пример просмотра стандартных прав доступа и расширенных ACL для файла в Linux

В Windows аналогичную роль играют ACL файловой системы NTFS. Каждому объекту соответствует дескриптор безопасности, включающий дискреционный список DACL и системный список SACL. DACL определяет, что разрешено и запрещено, а SACL

используется для аудита обращений к объекту (Microsoft 2025c). Наследование записей доступа от родительских каталогов значительно облегчает администрирование, но одновременно создаёт риск непреднамеренного расширения полномочий, если наследуемые правила настроены слишком широко (Microsoft 2025a).

Визуальный пример просмотра и настройки прав доступа к файлу в Windows приведён на рис. 3.

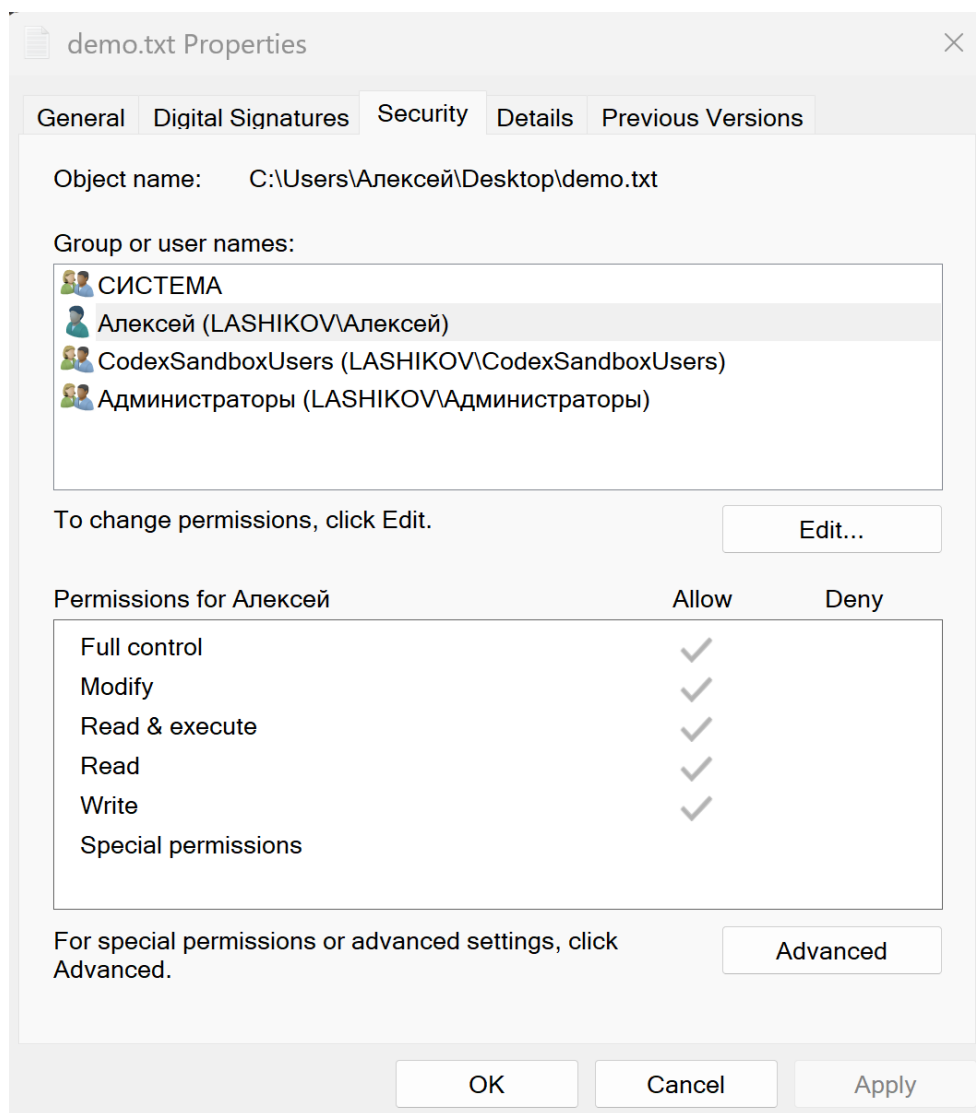


Рисунок 3: Пример просмотра и настройки прав доступа к файлу в Windows на вкладке «Безопасность»

3.2. Обязательные ограничения и песочницы

Одних только файловых прав часто недостаточно. Если вредоносный код запускается от имени пользователя, уже имеющего доступ к документу, обычный DAC не сможет этому помешать. Поэтому в современных ОС применяются обязательные ограничения.

В Windows такой механизм реализован в виде Mandatory Integrity Control. Он дополняет дискреционный доступ и проверяет уровень целостности субъекта и объекта до анализа DACL ([Microsoft 2025d](#)). Это означает, что даже при формально совпадающих правах запись в объект может быть запрещена, если уровень целостности процесса ниже требуемого.

В Linux и ряде UNIX-подобных систем для обязательного контроля применяются SELinux и AppArmor. SELinux использует метки безопасности и политики типа type enforcement, разрешая только те взаимодействия, которые явно предусмотрены политикой ([Fedora Project 2025](#)). AppArmor, напротив, ориентируется на профили для конкретных программ и ограничения по файловым путям ([Ubuntu 2026](#)). Оба механизма усиливают защиту файлов, так как снижают вероятность злоупотребления правами процесса после компрометации приложения.

В мобильных ОС идея обязательных ограничений выражена ещё сильнее. Android опирается на модель sandbox для приложений, уникальные UID и SELinux в режиме enforcing ([Android Open Source Project 2025a, 2025b](#)). В iOS сторонние приложения работают в изолированных контейнерах и не могут произвольно читать файлы других приложений без посредничества системных сервисов ([Apple 2025a](#)).

Принцип изоляции приложений в мобильных ОС иллюстрирует [рис. 4](#).

3.3. Принятие решения о доступе

На практике доступ к файлу разрешается не одной проверкой, а последовательностью действий. Сначала система сопоставляет пользователя с его учётной записью, затем анализирует обязательные ограничения, потом дискреционные права и, если требуется, записывает событие в журнал. Упрощённая схема показана на [рис. 5](#).

Такая последовательность показывает, почему даже корректно настроенные права владельца не гарантируют полный доступ: итоговое решение зависит и от дополнительных политик безопасности, и от режима аудита, и от состояния сеанса.

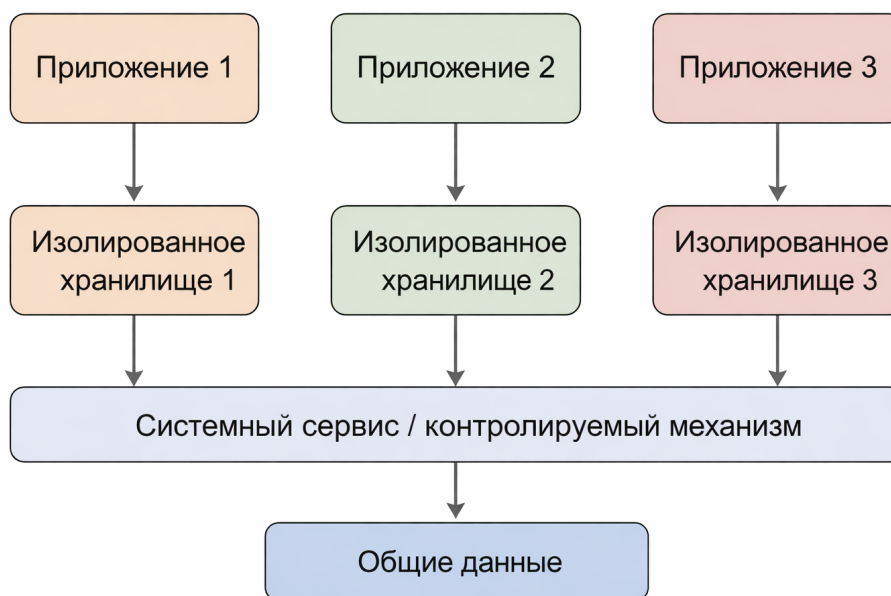


Рисунок 4: Схема изоляции приложений и файловых данных в мобильной операционной системе

4. Шифрование как метод защиты файлов

4.1. Назначение шифрования

Шифрование предназначено прежде всего для защиты данных в состоянии хранения, то есть *at rest*. Если устройство выключено, потеряно или диск извлечён из компьютера, злоумышленник не сможет прочесть содержимое файлов без ключей дешифрования (Microsoft 2025e; Android Open Source Project 2025b; Apple 2025b). Однако шифрование не решает всех проблем. Если система уже разблокирована и вредоносный процесс работает от имени пользователя, файл после расшифровки становится доступен в рамках имеющихся полномочий. Следовательно, шифрование необходимо рассматривать как один из слоёв защиты, а не как универсальное средство.

4.2. Полное и файловое шифрование

Существует несколько основных подходов к шифрованию. Полное шифрование диска или устройства защищает целый том и особенно эффективно против кражи носителя. В Windows для этого применяется BitLocker, в macOS — FileVault, в Linux — dm-crypt/LUKS (Microsoft 2025e; Apple 2025b; The Linux Kernel Documentation 2024).

Файловое шифрование работает на уровне отдельных файлов или каталогов. Примером

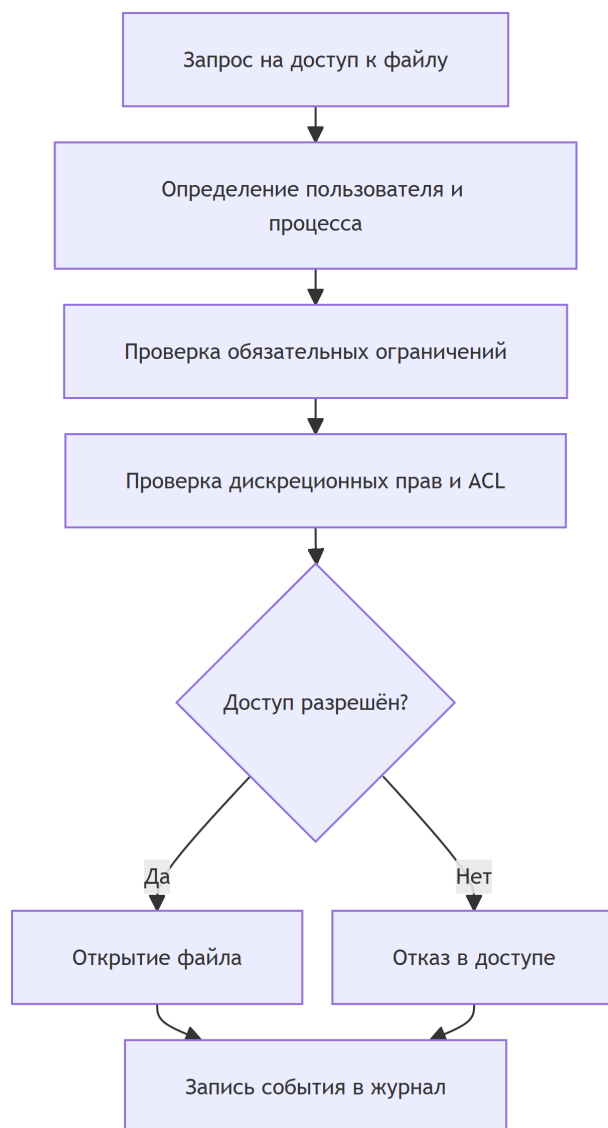


Рисунок 5: Упрощённая схема принятия решения о доступе к файлу

служит EFS в Windows, а также file-based encryption в мобильных системах. Такой подход удобен тем, что разные категории данных могут защищаться разными ключами и разблокироваться независимо ([Microsoft 2025b](#); [Android Open Source Project 2025b](#)).

Особое значение файловое шифрование имеет для Android. Официальная документация указывает, что Android 7.0 и выше поддерживает file-based encryption, а для новых устройств на Android 10 и выше именно FBE является обязательной моделью ([Android Open Source Project 2025b](#)). Это связано с тем, что разные файлы и профили пользователя могут разблокироваться отдельно, а часть сервисов остаётся доступной ещё до полного ввода пароля через механизм Direct Boot ([Android Open Source Project 2025b](#)).

На [рис. 6](#) показан пример экрана восстановления BitLocker в Windows. Такой механизм демонстрирует, что данные на диске хранятся в зашифрованном виде, а доступ к ним возможен только после успешной аутентификации и ввода соответствующего ключа восстановления.

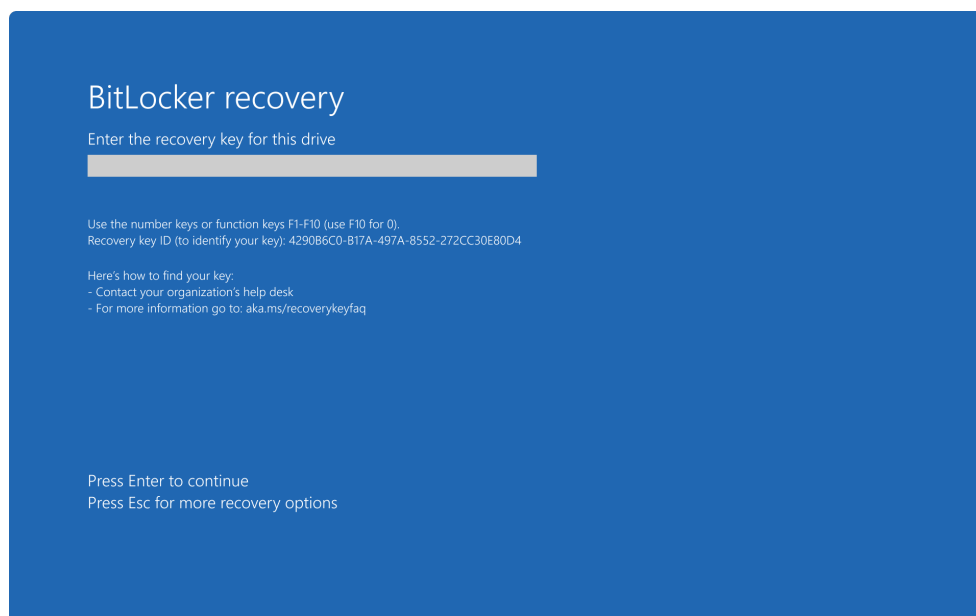


Рисунок 6: Пример запроса ключа восстановления BitLocker при доступе к зашифрованному диску

4.3. Аппаратная поддержка ключей

Эффективность шифрования во многом зависит от того, как именно хранятся и защищаются ключи. Современные ОС всё чаще опираются на аппаратный корень доверия: TPM в персональных компьютерах, Secure Enclave в устройствах Apple, аппаратно защищённый Keystore в Android ([Microsoft 2025e](#); [Android Open Source Project 2025c](#); [Apple 2025a](#)). Это усложняет извлечение ключей и повышает устойчивость к атакам на загрузочную цепочку.

5. Контроль целостности и аутентичности

5.1. Проверка целостности данных

Защита файлов должна обеспечивать не только запрет чтения, но и обнаружение несанкционированных изменений. Для этого применяются контрольные суммы, хэширование, цифровые подписи и механизмы доверенной загрузки.

На уровне файловых систем важны встроенные средства контроля целостности. Некоторые современные файловые системы используют контрольные суммы для данных и метаданных, что позволяет обнаруживать повреждения и в отдельных случаях автоматически восстанавливать корректную копию из зеркала или другой избыточной структуры ([The Linux Kernel Documentation 2024](#)). Хотя подобные механизмы не всегда подтверждают, кто именно изменил данные, они позволяют вовремя заметить нарушение целостности.

5.2. Подписи и доверие к исполняемым файлам

Для системных и исполняемых файлов большое значение имеют цифровые подписи. В Windows широко используется Authenticode, в macOS — code signing и notarization, в Android установка приложения без подписи невозможна ([Microsoft 2025e](#); [Apple 2025a](#); [Android Open Source Project 2025a](#)). Подпись помогает убедиться в происхождении файла и в том, что его содержимое не было изменено после подписания.

Целостность тесно связана и с доверенной загрузкой. Если злоумышленник получает возможность подменить загрузчик, драйвер или ядро, то он потенциально обходит и файловые права, и политики MAC. Поэтому механизмы Secure Boot, Measured Boot и аналогичные цепочки доверия играют важную роль в общей защите файлов, особенно системных ([Microsoft 2025e](#); [Apple 2025a](#)).

6. Аудит и журналирование

Даже хорошо настроенные права доступа и шифрование не исключают инциденты. Поэтому важным методом защиты файлов является аудит: система фиксирует факты обращения к объектам, изменения прав, неудачные попытки доступа и иные события безопасности.

В Windows аудит файлового доступа зависит от двух условий: соответствующая политика должна быть включена, и у конкретного объекта должен быть настроен SACL ([Microsoft 2025c](#)). В Linux ту же задачу решают auditd и правила аудита, позволяющие отслеживать обращения к важным файлам и каталогам ([Ubuntu 2026](#)). В результате администратор

получает возможность не только заметить подозрительные действия, но и восстановить картину событий после инцидента.

Журналирование особенно важно для критичных системных файлов, конфигураций и секретов. Если доступ к ним был получен правомерно с точки зрения DAC, но необычно с точки зрения поведения пользователя или процесса, именно журнал позволяет обнаружить аномалию. Поэтому аудит нельзя считать второстепенным дополнением: он является самостоятельным уровнем защиты.

7. Безопасное удаление и остаточные данные

Обычное удаление файла, как правило, не уничтожает его содержимое немедленно. Чаще всего операционная система лишь помечает соответствующие блоки как свободные, а сами данные ещё некоторое время могут быть восстановлены. Из-за этого в средства защиты файлов включают методы безопасного удаления и санитизации носителей.

Современным ориентиром в этой области является NIST SP 800-88 Rev. 2. Документ выделяет уровни Clear, Purge и Destroy и подчёркивает, что выбор метода зависит от типа носителя и требуемой степени защиты ([Chandramouli et al. 2025](#)). Для современных SSD простая многократная перезапись уже не считается универсально надёжным подходом, поскольку контроллер накопителя и механизмы wear leveling не гарантируют перезапись всех физических областей хранения ([Chandramouli et al. 2025](#)).

Следовательно, безопасное удаление нужно рассматривать не как одну команду, а как совокупность организационных и технических мер. Для жёстких дисков, SSD, мобильных устройств и виртуальных машин методы могут существенно различаться.

8. Сравнение подходов в различных операционных системах

Сопоставить рассмотренные методы удобно в обобщённой таблице. В [табл. 1](#) показано, какие решения чаще всего используются в разных семействах ОС.

Таблица 1: Сравнение основных методов защиты файлов в разных семействах операционных систем

Семейство ОС	Контроль доступа	Доп. механизмы	Шифрование	Краткая характеристика
Windows	NTFS DACL/SACL, ACE	MIC	BitLocker, EFS	Гибкая ACL-модель и развитый аудит
Linux	POSIX-права, POSIX ACL	SELinux, AppArmor	LUKS, fscrypt	Гибкая настройка, важна корректная конфигурация
macOS	BSD-права, ACL	Sandbox, Gatekeeper	FileVault	Сильная связка доступа, подписи и аппаратной защиты
Android	UID-изоляция, sandbox	SELinux	FBE	Акцент на контейнеризации приложений
iOS/iPadOS	Контейнеры приложений	Системная песочница	Data Protection	Закрытая модель с контролируемым доступом

Из табл. 1 видно, что различия между системами связаны не столько с наличием или отсутствием конкретных механизмов, сколько с их композицией. Windows делает акцент на развитой ACL-модели и интеграции с корпоративной инфраструктурой. Linux предоставляет очень гибкий набор средств, но требует аккуратного администрирования. Мобильные ОС сильнее опираются на песочницы, аппаратную защиту ключей и строгую изоляцию приложений.

9. Проблемы и ограничения существующих методов

Несмотря на большое число средств защиты, на практике уязвимости нередко возникают из-за ошибок конфигурации и человеческого фактора. Слишком широкие наследуемые ACL, отключение SELinux или AppArmor ради совместимости, потеря расширенных атрибутов при копировании файлов, неверно настроенный аудит, хранение ключей восстановления в небезопасном месте — всё это снижает реальную эффективность защиты ([Fedora Project 2025](#); [Ubuntu 2026](#); [Базальт СПО 2025](#)).

Кроме того, каждый метод имеет свои ограничения. Права доступа бессильны против администратора или уязвимого ядра. Шифрование бесполезно, если система уже разблокирована и атакующий действует от имени пользователя. Контроль целостности может обнаружить подмену, но не всегда предотвратит её. Аудит полезен для расследования,

но не блокирует действие в моменте. Безопасное удаление осложняется особенностями современных носителей и виртуализированных сред (Chandramouli et al. 2025).

Именно поэтому в реальных системах важен принцип глубокой обороны: разные методы должны не заменять, а дополнять друг друга.

10. Заключение

В ходе рассмотрения темы было установлено, что защита файлов в операционных системах представляет собой комплекс взаимосвязанных методов. Базовым уровнем остаётся управление доступом — традиционные права, ACL, роли и политики. Дополнительно используются обязательные ограничения и песочницы, которые уменьшают ущерб от компрометации приложений. Шифрование защищает данные при физической утрате носителя, контроль целостности помогает выявлять подмену и повреждение файлов, аудит обеспечивает наблюдаемость, а безопасное удаление уменьшает риск восстановления уже удалённой информации.

Главный вывод состоит в том, что универсального средства защиты файлов не существует. Надёжность обеспечивается только сочетанием нескольких слоёв: корректной аутентификации, разумного разграничения прав, обязательных политик, шифрования, контроля целостности, аудита и продуманной работы с жизненным циклом данных. Для администратора и пользователя это означает, что реальная безопасность зависит не только от наличия функций в ОС, но и от качества их настройки и сопровождения.

Список литературы

- Android Open Source Project. 2025a. “App Sandbox.” 2025. <https://source.android.com/docs/security/app-sandbox>.
- . 2025b. “File-Based Encryption.” 2025. <https://source.android.com/docs/security/features/encryption/file-based>.
- . 2025c. “Hardware-Backed Keystore.” 2025. <https://source.android.com/docs/security/features/keystore>.
- Apple. 2025a. “Apple Platform Security.” 2025. https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf.
- . 2025b. “Volume Encryption with FileVault in macOS.” 2025. <https://support.apple.com/guide/security/volume-encryption-with-filevault-sec4c6dc1b6e/web>.
- Astra Linux. 2026a. “Дискреционное Управление Доступом.” 2026. <https://docs.astralinux.ru/latest/szi/szi/dac/>.

- . 2026b. “Мандатное Управление Доступом.” 2026. <https://docs.astralinux.ru/latest/szi/szi/mac/>.
- . 2026c. “Мандатный Контроль Целостности.” 2026. <https://docs.astralinux.ru/latest/szi/szi/mic/>.
- Astra Linux Wiki. 2021. “Руководство По КСЗ. Часть 1.” 2021. https://wiki.astralinux.ru/download/attachments/137563555/Ruk_KSZ_1.pdf?api=v2&modificationDate=1635171969446&version=1.
- . 2026. “Списки Управления Доступом к Файловым Объектам (ACL).” 2026. <https://wiki.astralinux.ru/pages/viewpage.action?pageId=137567873>.
- Chandramouli, Ramaswamy et al. 2025. “Guidelines for Media Sanitization.” NIST Special Publication 800-88 Revision 2. National Institute of Standards; Technology. <https://csrc.nist.gov/pubs/sp/800/88/r2/final>.
- Fedora Project. 2025. “SELinux Getting Started.” 2025. <https://docs.fedoraproject.org/en-US/quick-docs/selinux-getting-started/>.
- Hu, Vincent C., David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2019. “Guide to Attribute Based Access Control (ABAC) Definition and Considerations.” NIST Special Publication 800-162. National Institute of Standards; Technology. <https://csrc.nist.gov/pubs/sp/800/162/upd2/final>.
- Microsoft. 2025a. “ACE Inheritance Rules.” 2025. <https://learn.microsoft.com/en-us/windows/win32/secauthz/ace-inheritance>.
- . 2025b. “File Encryption.” 2025. <https://learn.microsoft.com/en-us/windows/win32/fileio/file-encryption>.
- . 2025c. “Icacs.” 2025. <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/icacs>.
- . 2025d. “Mandatory Integrity Control.” 2025. <https://learn.microsoft.com/en-us/windows/win32/secauthz/mandatory-integrity-control>.
- . 2025e. “Operating System Security: Encryption and Data Protection.” 2025. <https://learn.microsoft.com/en-us/windows/security/book/operating-system-security-encryption-and-data-protection>.
- National Institute of Standards and Technology. 2025a. “Discretionary Access Control.” 2025. https://csrc.nist.gov/glossary/term/discretionary_access_control.
- . 2025b. “Mandatory Access Control.” 2025. https://csrc.nist.gov/glossary/term/mandatory_access_control.
- . 2025c. “Role Based Access Control.” 2025. <https://csrc.nist.gov/projects/role-based-access-control>.
- The Linux Kernel Documentation. 2024. “Fscrypt.” 2024. <https://www.kernel.org/doc/html/v6.9/filesystems/fscrypt.html>.

Ubuntu. 2026. “AppArmor.” 2026. <https://ubuntu.com/server/docs/how-to/security/apparmor/>.

Базальт СПО. 2024. “Команда Getfacl.” 2024. <https://docs.altlinux.org/ru-RU/alt-server/10.2/html/alt-server/ch65s07.html>.

———. 2025. “Команда Setfacl.” 2025. <https://docs.altlinux.org/ru-RU/alt-server/11.1/html/alt-server/setfacl.html>.